AMATH445 project - SINDy and UDE analysis when trained on noise

Dashiel Balnaves

March 2026

UDE-and-SINDy-analysis-on-damped-oscillator

1 Background

When we have normal undamped oscillation we are simply able to plot solutions using the level curves of the Hamiltonian, but what if we make the system slightly more complicated and add a dampening term, and we want to analyze the system given by,

$$F = -x + \beta x^3 - \gamma x' \tag{1}$$

$$\begin{cases} x' = y \\ y' = -x + \beta x^3 - \gamma y \end{cases}$$

Further analysis shows that this system is clearly not Hamiltonian as total energy is not conserved. We expect stability at the equilibrium point $(0, 0)$, and then 2 saddles bounding solution curves that converge to this point.
We can use models such as Neural networks or symbolic regression to predict the differential equation given some data, through universal differential equations (Rackauckas and Yingbo Ma (2020)) and SINDy (Brunton et al. (2016)), but which could be better? And what can we do to get an accurate model?

1.1 Universal differential equation

A universal differential equation combines a known physics component of our system with an unknown part. For this system in particular we will consider the dampening component to be the unknown part.
Universal differential equations can model our systems dynamics (ie. learn the differential equation) with the following formulation.

$$\frac{du}{dt} = f_{known}(u, t; \beta) + f_\theta(u, t)$$

θ represents the parameter of our neural network to be learned. First we define our hybrid model, then we use the following loss function,

$$\mathcal{L}(\theta) \ = \frac{1}{n} \sum_{i=1}^{n} ||x_{model} - x_{data}||^2$$

Then we perform backpropagation through the loss function as we normally would in a neural network in order to update the parameter, note that the data is noisy.
We expect the UDE to work best if we keep the Neural network component small, initialize the

model close to 0, and regularize the NN output.
We will model with the following,

$$\begin{cases} x' = y \\ y' = -x + \beta x^3 - y g_\theta(x, y) \end{cases}$$

The reason we use the neural network to learn the unknown parameter and not the velocity is because then we ensure that the dampening is always proportional to the velocity and not position, and that the energy is being dissipated. If we did not do this we could get an output increasing energy at some epoch, which wouldn't make physical sense.

1.2 Symbolic regression with SINDy

Symbolic regression involves determining a function that relates the input-output data, typically using genetic programming, this can be applied to dynamical systems. (Brunton et al. (2016))
During the regression, few of the terms in the regression are important so we have a feature matrix Θ, and a sparse vector ξ which determines a linear combination of columns of Θ, so we have measurement,

$$\mathbf{y} = \Theta\xi$$

And then we get the regression problem (with L1 regularization added, due to the sparsity of ξ) given by,

$$\xi = \arg\min_{\xi'} ||\Theta\xi' - \mathbf{y}||_2^2 + \lambda||\xi'||_1$$

We can actually apply this to non-linear dynamics, by considering our library of symbols Θ, to have columns representing different mathematical symbols, of which there are many but few are important. And then ξ represents how important each symbol is.
For inputs at $\{t_1, ..., t_n\}$ for the system $\dot{\mathbf{x}} = f(\mathbf{x}(t))$. We have,

$$\mathbf{X} = \begin{pmatrix} \mathbf{x^T}(t_1) \\ \mathbf{x^T}(t_2) \\ ... \\ \mathbf{x^T}(t_n) \end{pmatrix}$$

And then based on this we can write a library matrix,

$$\Theta(\mathbf{X}) = \begin{bmatrix} | & | & | & ... & | & | & ... \\ \mathbf{1} & \mathbf{X} & \mathbf{X^2} & ... & \sin(\mathbf{X}) & \cos(\mathbf{X}) & ... \\ | & | & | & ... & | & | & ... \end{bmatrix}$$

Here you can see each dimension column represents a different symbol/function that can be arbitrarily chosen. But ultimately the model will try to learn the solution based on whatever you choose the functions representing the solutions to be. If we consider our own system where $\mathbf{x} \in \mathbb{R}^2$, then we are trying to learn the system by solving the regression problem produced by,

$$\dot{\mathbf{X}} = \Theta(\mathbf{X})\xi,\ \xi \in \mathbb{R}^{(1+2m)\times 2},\ \Theta(\mathbf{X}) \in \mathbb{R}^{n\times(1+2m)}$$

This is if we have n data points, and choose m functions, with 1 added. Then the columns of ξ represent the coefficients of our dynamical system to be learned. It would ideally be very sparse, because we would have ideally a large library of functions.
Unlike the UDE this tries to learn a component of the whole whole DE, and it will be interesting to see how they compare under generalization, but ultimately this is very cool because we don't have to feed data into a long Neural network to get a result (Unlike a neural ODE), instead we have a literal differential equation.

2 Data generation

First we have to generate data which represents the actual dynamics. We will generate the data using a C++ script simulating the Runge-Kutta 4th order method, and exporting to a CSV file. We will have multiple CSVs, one which represents Noisy data, using Gaussian noise, and one which represents the respective exact data.
You can initialize the csv files yourself using a linux command line argument `./datagen filenam var h xinit yinit N.`
During the project, particularly for the UDE, with large time intervals, where there are many steps, performance is terrible. Results show that the loss using the ADAM (Kingma and Ba (2015)) optimizer stabilize after about 10×10^3 iterations, which if my original 500 data points were used, would've taken hours. Thus it should be trained under small intervals, making it limited in its generalizing ability potentially.
The reason the model is so slow is due to the fact it has to backsolve through the differential equation using the model each time in order to compute the total loss.
Both models will be trained on different levels of noisy data, and then finally we will do additional training on different initial conditions to see if the results improve. The various conditions I will use will be described below.
When reading this paper, to keep comparison fair, we train SINDy and the UDE on the same data, however SINDy is a lot faster in being trained and allows for greater flexibility.

3 Programming

One important aspect to note is that we are using the 'pysindy' library in order to do our symbolic regression. The generic np.gradient() does not suffice for this, which just performs standard differentiation to obtain $\dot{\mathbf{X}}$ using time intervals δt.
Instead we used SmoothedFiniteDifference() as part of the library, which uses the SavitzkyâGolay (Savitzky and Golay (1964)) filter, a least squares regression algorithm on our data by default. which aims to solve,

$$\min_{a_0,\ldots,a_k} \sum_{j=-m}^{m} \left(x_{i+j} - \sum_{r=0}^{k} a_r (t_{i+j} - t_i)^r \right)^2$$

Using the fitting polynomial for our data,

$$x(t) \approx a_0 + a_1 t + a_2 t^2 + \ldots$$

Additionally we used the library of non-linear functions consisting of up to order 3 polynomials, and trigonometric functions, suitable choices for our damped oscillator.
Another important thing about the SINDy model is that it uses the STLSQ (sequential threshold least squares algorithm) algorithm when trying to optimize our outputs ξ. It involves applying our usual least squares regression, then we keep only the coefficients of ξ such that for some λ, $|\xi_i| \leq \lambda$, since we expect ξ to be sparse. Then it is repeated $N-1$ times, a form of cross-validation when using different values of λ. Compared to another basic method such as LASSO, STLSQ is very good in resisting noise, which we will need.

3.1 Neural network layers

The neural network utilities 1 hidden linear layer, the first using 32 nodes for outputs, and then second linear layer converting the 32 outputs of the first layer to a single output representing the systems interactive coefficient.
Additionally, each layer uses an activation function to produce a non-linear result. The first layer uses a $Tanh(x)$ activation function. The reason for this is because the derivatives within our system are continuous, and require smooth vector fields in $\mathbb{R}^2$. Since $Tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$ satisfies this, it is appropriate, as apposed to ReLu, which could be highly unstable.
Additionally, Softplus is used on the output since $Softplus(x) = \log(1 + e^x)$ ensures that $g_\theta(x) \geq 0$, which if not enforced could cause system to explode, since we want the physical constraint of positive dampening.

The reason for such a shallow network is because for the purposes of this project we are comparing a neural network with learned physics, and SINDy, thus I want to be able to build the network relatively quickly. Adding too many layers just becomes too computationally expensive, which as we will see effects our UDE substantially.

3.2 Training the UDE and SINDy

The core of this project is to see how our models respond to being trained on varying levels of gaussian noise applied to our training data.
We will use the following levels of standard deviation for Noise, and decide to fix β, γ.

$$\sigma = 0 \text{ (no noise)} \qquad \beta = 1$$
$$\sigma = 0.01 \text{ (small noise)} \qquad \gamma = 0.2$$
$$\sigma = 0.05 \text{ (medium noise)}$$
$$\sigma = 0.1 \text{ (large noise)}$$

Our C++ script using the Runge-Kutta 4th order method will generate the data needed to train the models, and ultimately I will directly compare the trained data, true data, and predicted data trajectories within a phase portrait.
Of course however the UDE and SINDy ultimately achieve different things, and thus metrics to measure include whether or not SINDy successfully recovered the original differential equation, and also whether the UDE successfully learned $\gamma = 0.2$.
For both we can measure the mean square error on test data, with true data.
Another interesting aspect is that our data is trained on the interval $t \in [0, 5]$ with 100 equidistant time steps, and initial condition $\mathbf{x_0} = (0, 0.5)$ (due to performance limitations). Since using this we are trying to recover the original equations, we can test the models on time intervals with $t > 5$ and different initial conditions, and see if the models break down, or suffer from poor generalization. This will be part of separate tests however.

4 Results with discussion

First we train the UDE on our noisy data of different levels, at first I started with the learning rate for our ADAM optimizer of $\lambda = 1 \times 10^{-3}$. We observe the following training loss curves.
And after comparing the results in a phase portrait we see that as the noise increases, the scale decreases. Test data was taken as data points in time increments, $\delta t = 0.07$, $N = 200$ with the same initial conditions.

Even at $\sigma = 0$, we see that the model overestimates the dampening term, which is what causes the correct shape, but incorrect scale. This represents the bias of the MSE based model towards overdamped trajectories, which is amplified under noisy conditions, but instead is slightly under damped. In other words the model decreases dampening to account for the MSE loss, even with the small amount at no noise added.

One possible solution is a decreased learning rate, or even training the model at different initial conditions, to prevent the bias towards this one curve, and increase generalization. We will train on another initial condition, which should help the model learn the system globally to get better results beond the given time interval.
We get the following functions when training the SINDy model on the data with the python pysindy library. With this model we do in fact use the Fourier library as part of our library, which requires adding an additional `integrator="odeint"` parameter in the `.simulate()` function to speed up processing. We use the library because we want the flexibility, since oscillatory motion could imply the presence of trigonometric functions.
we get the following equations generated by pySindy,

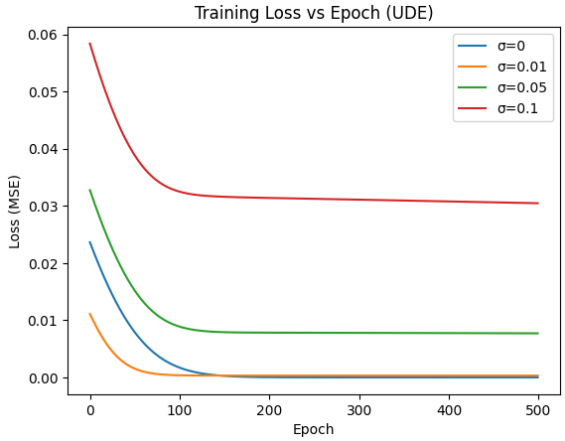

Figure 1: UDE loss, lr $= 1 \times 10^3$

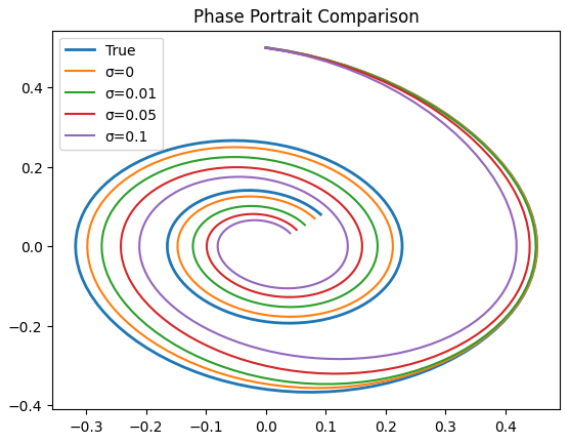

Figure 2: UDE trajectories at different noise levels

```
    Training with noise Ï = 0
    Discovered system:
    (x0)' =  1.006 x1 + -0.006 sin(1 x1)
    (x1)' = -0.565 x0 + -0.192 x1 +  0.926 x0^3 +  0.002 x0 x1^2 + -0.435 sin(1
x0) + -0.008 sin(1 x1)
    Training with noise Ï = 0.01
    Discovered system:
    (x0)' =  1.307 x1 + -0.317 sin(1 x1)
    (x1)' = -297.947 x0 +  0.359 x1 +  0.044 x0^2 +  49.680 x0^3 + -0.075 x0 x1^2
+  296.991 sin(1 x0) + -0.565 sin(1 x1)
    Training with noise Ï = 0.05
    Discovered system:
    (x0)' =  1801.746 x1 + -1.032 x0 x1 +  0.807 x1^2 + -2.094 x0 x1^2 + -297.742
x1^3 + -1800.577 sin(1 x1)
    (x1)' = -890.928 x0 +  0.616 x0^2 + -2.197 x0 x1 +  0.413 x1^2 +  148.094
x0^3 +  4.315 x0^2 x1 +  889.573 sin(1 x0)
    Training with noise Ï = 0.1
    Discovered system:
    (x0)' = -103.520 1 + -682.511 x0 +  3044.746 x1 +  0.321 x0^2 + -1.772 x0 x1
+  51.896 x1^2 +  111.763 x0^3 + -0.284 x0^2 x1 + -0.511 x0 x1^2 + -500.287 x1^3
+  682.409 sin(1 x0) + -3043.847 sin(1 x1) +  103.512 cos(1 x1)
    (x1)' =  372.623 1 + -3525.003 x0 + -181.000 x0^2 + -3.313 x0 x1 +  0.857
x1^2 +  577.781 x0^3 +  8.156 x0^2 x1 + -1.880 x0 x1^2 +  3524.133 sin(1 x0) +
-372.799 cos(1 x0)
```

It is quite obvious in this case that the solution diverges quite significantly from the differential at higher levels of noise, with the system completely breaking down outside training window. This is possibly due to the shorter training window causing prominence in the Fourier functions. Notice even at $\sigma = 0$, we get some slight weights in the Fourier functions, but represents the original system very well. Indeed we see this in the phase portraits. We can see that SINDy has adopted

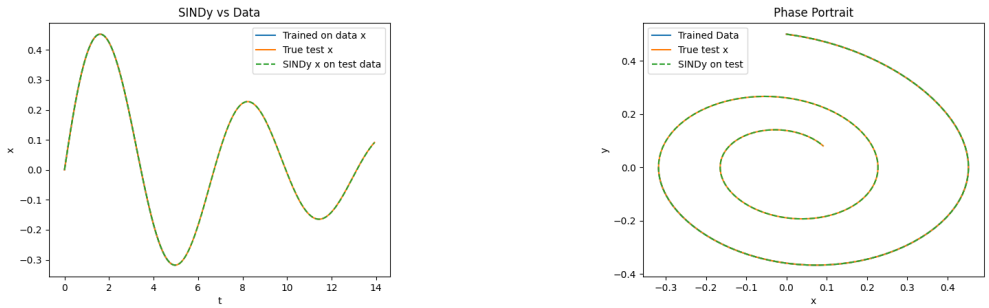

Figure 3: SINDy with fourier library included, $\sigma = 0$

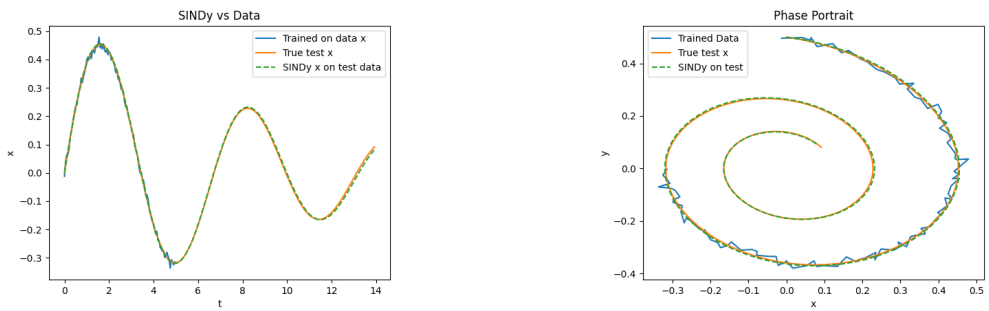

Figure 4: SINDy with fourier library included, $\sigma = 0.01$

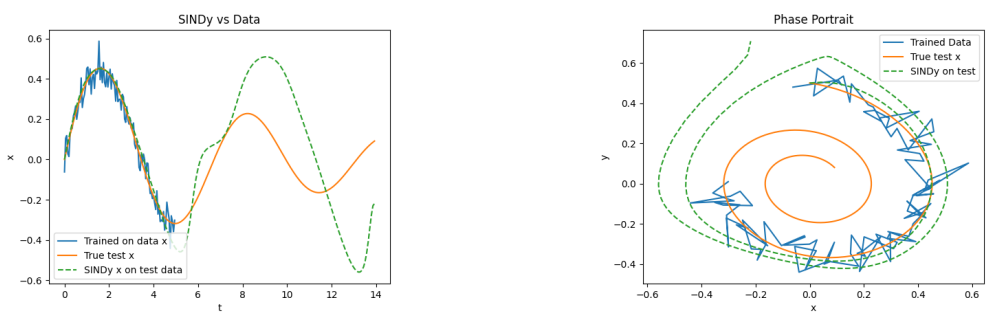

Figure 5: SINDy with fourier library included, $\sigma = 0.05$

the equation to the data very well, however going beyond the time interval that the training data exists on causes various issues with the noisy data particularly. For the noisy data, the system completely blows up. This is also caused by the oscillatory nature of the system.
Unlike the UDE however where it is best to keep our time interval small for performance purposes, it isn't unreasonable to train SINDy on a time interval that is much larger and inclusive of multiple periods, but for the purposes of this project, we keep the training the same as the UDE. Another option is to discard the Fourier library, and only include polynomials up to degree 3, but if the differential equation were unknown, this would not be an appropriate assumption.
One other solution if we do really insist on training on a smaller interval with the Fourier library,

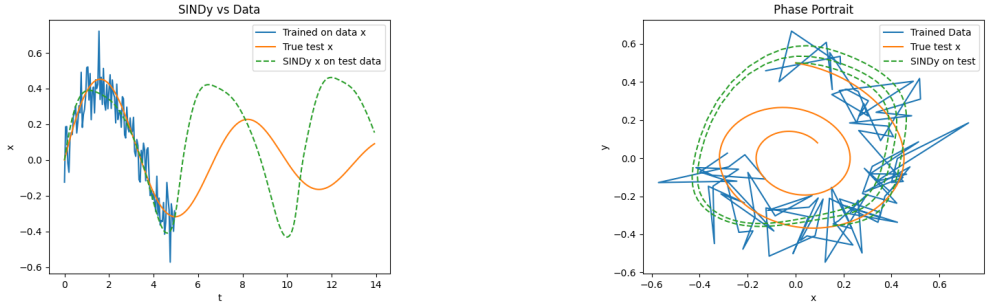

Figure 6: SINDy with fourier library included, $\sigma = 0.1$

is to do additional training on both models with another initial condition. Since even for SINDy, changing the library is not enough as we don't even reach a full period in our training data, so the model does not know the solution dampens, it knows it is periodic however in both cases.
We will choose $\mathbf{x_0} = (-1, 0.5)$, which should help confirm the symmetry of the system, using the original time steps. We do however have to alter both models for this to work though. We want to try to understand the system globally.
For the Neural network involved in the UDE we cannot simply train the model on our original data and then train on the new training set, since the neural network learns sequentially. Instead we need to train a new model on both sets combined, and randomly sample from both of them using `random.choice(samples)`. One downside is that due to the increased complexity of the model, we require more training iterations, thus we use 1000 epochs (which took 21 minutes to train), instead of our original 500.

For SINDy, we also need to make similar adjustments, though not the same. If we simply re-

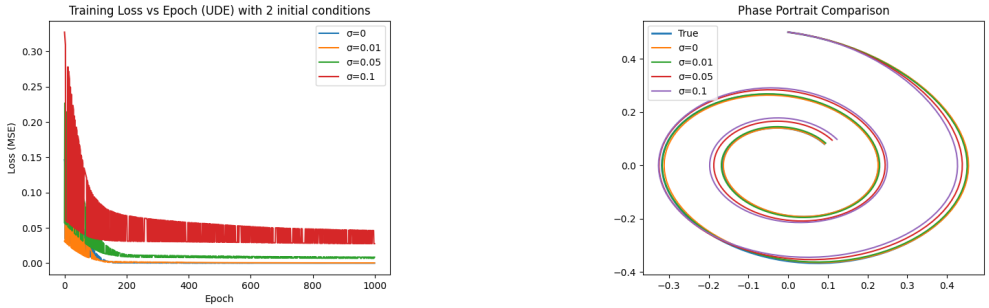

Figure 7: The UDE trained on 2 separate initial conditions, greater complexity

train the old model on the new data, this causes a physical discontinuity in the system. SINDy's derivatives cannot be discontinuous, but doing this will cause them to be, making the model highly unstable.
Instead we fit the model by using multiple datasets, which SINDy allows for using lists as arguments for fitting (`model.fit([X1,X2], t=[t1,t2], x_dot = [Xdot1,Xdot2])`) The results for SINDy and the UDE with both initial conditions are as follows. Since we already have a decent representation of SINDy with lower levels of noise, we will focus on higher levels for SINDy.

For the UDE, we can compare our final losses, When considering $\sigma = 0$ for example, the Mean squared error takes much longer to converge, with our 2 initial conditions taking 25 minutes, and 1 initial condition taking 12 minutes and is initially more unstable. And if we made the data even more complex for better generalization would require even more iterations. This explains why the original paper on UDEs uses 10000 iterations (Devgupta et al. (2024)) with the ADAM optimizer. The results of the model on the test data are as follows. We see that the result of the UDE is still very good when trained on an additional initial condition. Additional initial conditions would benefit greatly from greater computational time (which would take hours without GPU acceleration).

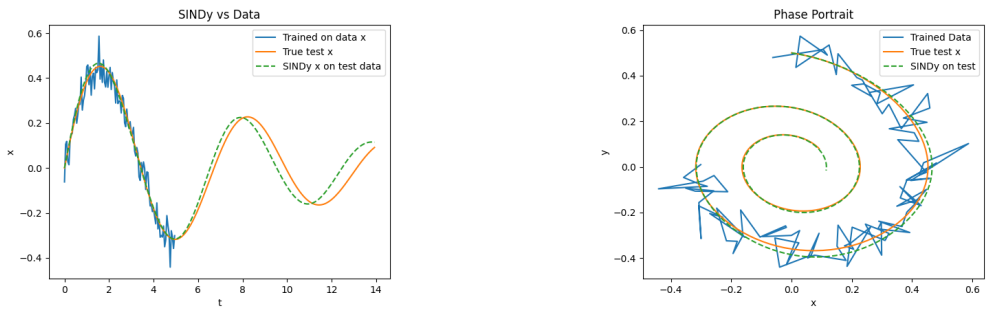

Figure 8: SINDy with fourier library included, $\sigma = 0.05$

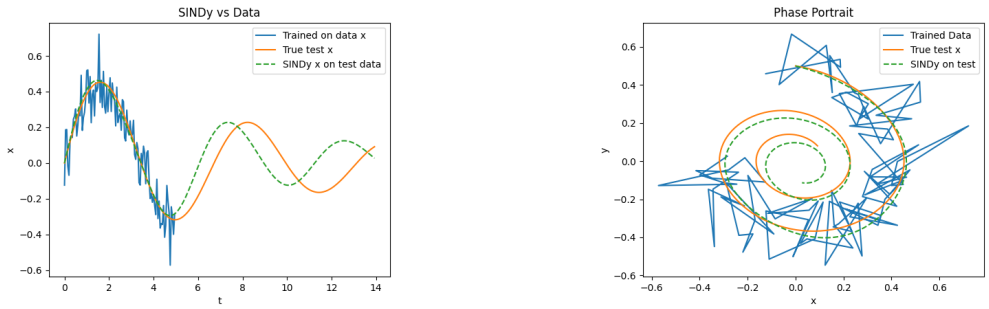

Figure 9: SINDy with fourier library included, $\sigma = 0.1$

Training Standard deviation	1 initial condition	2 initial conditions
0	`Epoch 490, Loss: 0.000000`	`Epoch 990, Loss: 0.000004`
0.01	`Epoch 490, Loss: 0.000284`	`Epoch 990, Loss: 0.000296`
0.05	`Epoch 490, Loss: 0.006733`	`Epoch 990, Loss: 0.006879`
0.1	`Epoch 490, Loss: 0.027419`	`Epoch 990, Loss: 0.027665`

Table 1: Final training MSE loss comparison for UDE

Training Standard deviation	1 IC MSE on test	2 IC MSE on test
0	`Test MSE=6.2182488136e-07`	`Test MSE=8.1935568232e-06`
0.01	`Test MSE=2.60269666796e-06`	`Test MSE=8.9651111920e-06`
0.05	`Test MSE=0.000338970101438`	`Test MSE=0.000314502685796`
0.1	`Test MSE=0.00093013053992`	`Test MSE=0.0007825251086`

Table 2: UDE performance on test data

We see this in the fact that even at $\sigma = 0$, after 1000 epochs for 2 IC's, the loss is still greater than that of 1 after 500 epochs. An approach that Devgupta et al. (2024) takes is using Rms-prop after 10000 iterations to stabilize the Mean squared error, and get a more reasonable result. But we can see in 7, that the benefit is that the model now performs better globally. Since the portion outside the trained time window performs much better even at higher noise levels, showing the power of multiple IC's.

On the other hand, SINDy works much better at the higher noise levels when considering 2 initial conditions. Although not perfect, it allows the sparse representation of dynamics to understand the library of functions actually in use. If trained on even more points, should work even better under generalization. Shockingly impressive improvement, but shows the power of a global understanding of the system.

5 Conclusion

In conclusion, we see that SINDy and the UDE both have their benefits, but without adjustments both models suffer under noise when trying to determine the system globally beyond training data. Generally it is easier for the UDE to have a global understanding of the system, due to the already learned physics. But if no information is known, given a long enough training window, SINDy is able to accurately reproduce the differential equation, with the same training data.
For this reason, without noise (or little) SINDy will perform significantly better beyond the training window, since the sparse representation of functions is an accurate representation of the system. But at higher levels of noise, the UDE at least remains stable and retains shape (though overestimating dampening), whereas the SINDy model is very inaccurate, and unstable beyond the training window. This is likely due to the flexibility of the functions used by SINDy.
However both models can be improved beyond the training window by training on multiple initial conditions for a better global understanding of the system. This is especially useful when utilizing SINDy, since it causes the model to retain the correct shape despite high levels of Gaussian noise. The UDE outside the training window also gives a result much closer to the correct solution.
From this however arises a big problem, as training complexity increases, the UDE requires substantially more computation. This becomes very time intensive due to how the loss function is computed. Methods such as GPU acceleration (especially for the Runge-Kutta calculations, which are the most time consuming) are being explored for this (Vermeire (2023)), but other methods to help the loss converge faster should be explored too. This isn't as much of a problem with SINDy however, showing the power of symbolic regression.
Noisy data is a big concern when it comes to modeling real world systems, and thus it is important to develop models that are resistant to high levels of noise. Ultimately for good generalization of global dynamics you want an appropriate training window, and a variety of starting conditions. More complicated differential equations require this especially since behavior outside of known data becomes unpredictable to basic human intuition.

References

Brunton, S. L., Proctor, J. L., and Kutz, J. N. (2016). Discovering governing equations from data by sparse identification of nonlinear dynamical systems. *Proceedings of the National Academy of Sciences*, 113(15):3932–3937.

Devgupta, R., Dandekar, R. A., Dandekar, R., and Panat, S. (2024). Scientific machine learning in ecological systems: A study on the predator-prey dynamics. *arXiv preprint arXiv:2411.06858*.

Kingma, D. P. and Ba, J. (2015). Adam: A method for stochastic optimization. In *3rd International Conference for Learning Representations, ICLR 2015*, San Diego, CA, USA.

Rackauckas, C. and Yingbo Ma, e. a. (2020). Universal differential equations for scientific machine learning. *CoRR*, abs/2001.04385.

Savitzky, A. and Golay, M. J. E. (1964). Smoothing and differentiation of data by simplified least squares procedures. *Analytical Chemistry*, 36(8):1627–1639.

Vermeire, B. C. (2023). Gpu accelerated paired explicit runge-kutta methods for high-order spatial discretizations. *Progress in Canadian Mechanical Engineering. Volume 6*, 6.